Contents

Specification Engineering Tools

I would expect the **Specification Engineer's toolchain to become a new layer of the software-development stack**—not just better requirements-management software.

The key shift is from:

> **documents about software**

to:

machine-readable artifacts that constrain, generate, and verify software.

1. The Specification Engineer's toolbox

I would organize the tools into nine categories:

Tool category	Purpose	Examples today	Likely evolution
Elicitation	Extract intent	interviews, product docs, LLMs	AI specification agents
Specification editors	Define structured behavior	Markdown, YAML, JSON Schema	Spec IDEs
Modeling	Model states/workflows	UML, state machines, BPMN	executable behavioral models
Contracts	Define interfaces	OpenAPI, Protobuf, JSON Schema	contract synthesis
Constraints	Define what must/mustn't happen	assertions, OCL, policies	constraint engines
Verification	Prove/test properties	unit tests, formal methods	automated spec verification
Evaluation	Test probabilistic behavior	eval frameworks, LLM judges	specification-derived evals
Traceability	Connect intent→implementation	Jira, DOORS, Git	live traceability graphs
Change management	Manage evolving specs	Git, PRs, reviews	semantic specification versioning

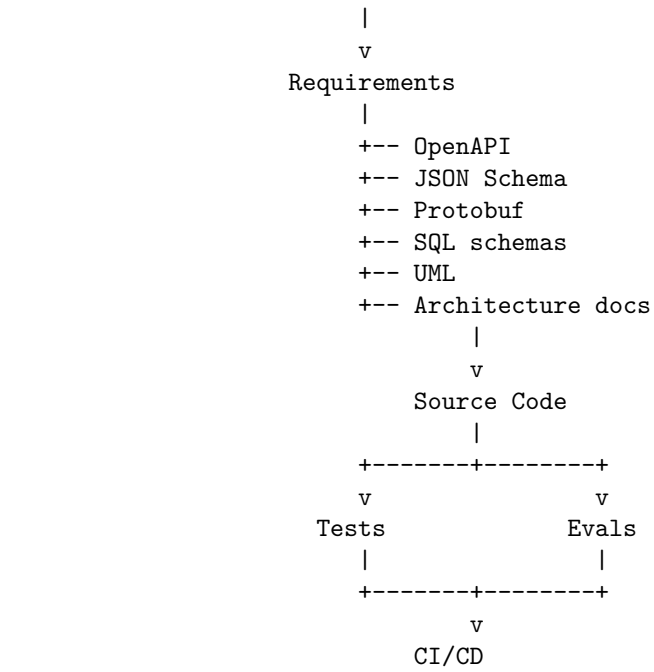
The interesting part is how these categories begin to **converge**.

---

2. The current toolchain is fragmented

Today, the Specification Engineer has to stitch together many tools:

```
Product
   |
   +-- Jira / Linear
   +-- Notion / Confluence
   +-- Product docs
```



The fundamental problem is that **these artifacts don't form one coherent specification graph**.

A requirement in Jira doesn't necessarily know which API contract implements it.

The API doesn't necessarily know which requirement motivated it.

The test doesn't necessarily know which invariant it verifies.

The LLM eval doesn't necessarily know which behavioral requirement it measures.

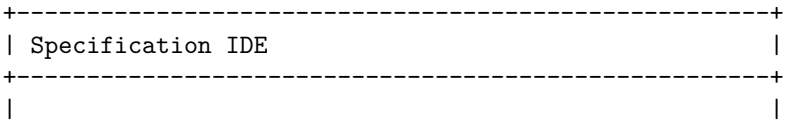
That's going to change.

---

3. The first major evolution: the Specification IDE

I think the most important new tool will be something analogous to today's IDE, but for **system behavior rather than source code**.

Imagine:



```
| Requirements                                          |
|   R1  Users can upload documents                      |
|   R2  Users can search documents                      |
|                                                       |
| Behavior                                              |
|   B1  Document ingestion workflow                     |
|   B2  Search workflow                                 |
|                                                       |
| Contracts                                             |
|   C1  POST /documents                                 |
|   C2  POST /search                                    |
|                                                       |
| Invariants                                            |
|   I1  User isolation                                  |
|   I2  Deleted documents never retrieved               |
|                                                       |
| Evals                                                 |
|   E1  Retrieval accuracy                              |
|   E2  Groundedness                                    |
|                                                       |
| Coverage                                              |
|   Requirements: 100%                                  |
|   Invariants: 100%                                    |
|   Evals: 94%                                          |
|                                                       |
+-------------------------------------------------------+
```

The editor would understand the semantics of the artifacts.

For example, if you change:

```
I1: Users cannot access another user's documents.
```

the system could immediately show:

```
Affected:
  API-C1
  Retrieval workflow B2
  Authorization policy P3
  Tests T17, T18
  Eval E7
  Agent permissions A2
```

That's much more powerful than editing a Word document.

---

4. Natural language will become an interface to the specification system

The Specification Engineer won't necessarily write every specification manually.

You might say:

> "The user should be able to share a document with another employee."

The specification agent responds:

> What constitutes an employee?
>
> Can access be revoked?
>
> Does revocation take effect immediately?
>
> Can the recipient reshare?
>
> What happens to existing cached content?
>
> Should sharing generate an audit event?

The agent is performing **requirements elicitation + ambiguity detection**.

You then approve the resulting structured specification.

This becomes:

```
Human intent
     ↓
Specification Agent
     ↓
Questions / ambiguities
     ↓
Human decisions
     ↓
Structured specification
```

This is likely to become one of the most important AI-native engineering workflows.

---

5. Specification languages will become more important

I don't think natural language alone is sufficient.

We'll increasingly see combinations of:

Natural language

Good for intent:

Users can share documents.

Structured schemas

Good for structure:

```yaml
share_document:
  actor: authenticated_user
  target: organization_user
  permission: read
```

Formal constraints

Good for invariants:

```
target.organization == document.organization
```

Executable assertions

Good for verification:

```
assert unauthorized_user_cannot_read(document)
```

Evaluation specifications

Good for probabilistic behavior:

```
groundedness >= 0.95
citation_accuracy >= 0.98
```

So the future specification language may be **polyglot**:

```
Natural language
       +
Schemas
       +
Contracts
       +
Constraints
       +
Executable tests
       +
AI evaluation criteria
```

---

6. Formal methods will move closer to mainstream development

This is another major evolution.

Historically, formal verification has been expensive and specialized.

Specification Engineering creates a natural place for it.

Instead of asking:

> "Can we formally verify the entire application?"

we ask:

> "Which properties are important enough to formally constrain?"

For example:

```
Security invariant:
    user A can never read user B's private document

Financial invariant:
    account balance cannot become negative

Workflow invariant:
    payment cannot be marked COMPLETED without authorization

Agent invariant:
    agent cannot execute privileged tool without approval
```

Tools such as **TLA+**, SMT/SAT solvers, model checkers, type systems, and policy engines can increasingly operate underneath the specification environment.

The Specification Engineer doesn't necessarily need to be a formal-methods specialist.

The AI can translate:

> "A payment cannot be captured twice."

into a formal property and ask the solver whether the modeled workflow permits a violation.

---

7. Contract tools will become generative rather than descriptive

Today OpenAPI, Protobuf, JSON Schema, etc. primarily describe interfaces.

Future systems will treat the contract as a **generative source**.

For example:

```
Specification
      ↓
API contract
```

```
      ↓
+-- server stubs
+-- client SDK
+-- validation
+-- test cases
+-- documentation
+-- monitoring rules
```

The Specification Engineer therefore spends less time writing boilerplate and more time defining **semantics**.

The same principle applies to database schemas, event schemas, policy definitions, and agent tool definitions.

---

8. The specification will generate the eval suite

This is particularly important for AI systems.

Suppose the specification says:

> The assistant must answer questions using only information contained in authorized documents.

The system should automatically derive tests such as:

```
1. Question answerable from authorized document
2. Question requiring unauthorized document
3. Question with conflicting documents
4. Question with no supporting evidence
5. Prompt injection inside a document
6. Question requiring multiple documents
7. Ambiguous question
```

Then:

```
Specification
      ↓
Test/eval generation
      ↓
Golden cases
      ↓
Adversarial cases
      ↓
Runtime evaluation
```

This creates an extremely important feedback loop:

> **The specification defines not only what the agent should do, but how the system determines whether the agent did**

it.

---

9. Traceability becomes a graph rather than a spreadsheet

I think this will be one of the biggest changes.

Instead of:

```
Requirement → ticket → code → test
```

you get a semantic graph:

```
                +-- API
                |
Requirement - Specification - Invariant
                |      |
                |      +-- Policy
                |
                +-- Agent behavior
                |
                +-- Implementation
                |
                +-- Test
                |
                +-- Eval
```

The system can then answer questions automatically:

> "Show me everything affected by changing this requirement."
>
> "Which requirements aren't covered by tests?"
>
> "Which implementation behavior isn't justified by a specification?"
>
> "Which specifications have contradictory constraints?"
>
> "Which eval failures correspond to specification violations?"

That is **semantic traceability** rather than project-management traceability.

---

10. Git will evolve from source control toward specification control

Today:

```
git diff
```

shows:

```
- timeout = 30
+ timeout = 10
```

A future specification-aware system might say:

```
Behavioral change detected:

Search timeout:
    30s → 10s

Affected specifications:
    S-17 Retrieval
    S-21 Availability

Affected guarantees:
    G-04 Search availability

Affected evaluations:
    E-12 timeout recovery

Potential violation:
    S-21 requires recovery within 15s.
```

That's a fundamentally richer notion of change management.

The important object isn't merely **what lines changed**, but:

> **What system behavior changed?**

---

11. AI agents themselves become Specification Engineering tools

Eventually I expect specialized agents:

Specification Analyst

Extracts requirements and identifies ambiguity.

Specification Architect

Turns requirements into behavioral models and contracts.

Constraint Engineer

Finds invariants and formalizes them.

Test/Eval Engineer

Generates verification from specifications.

Consistency Agent

Looks for contradictions.

Traceability Agent

Maintains requirement → specification → implementation → eval relationships.

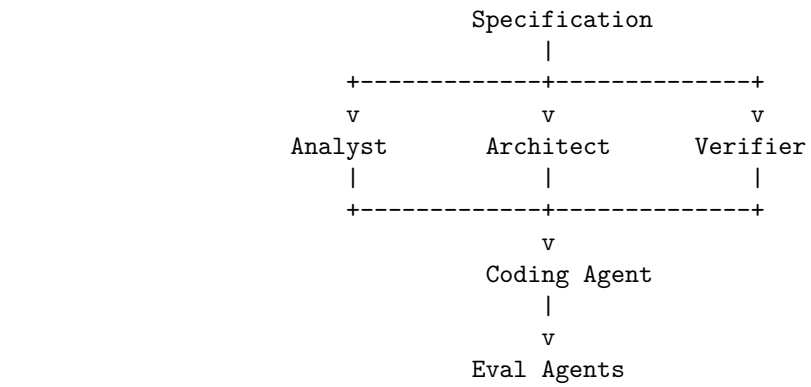
Change Impact Agent

Analyzes proposed specification changes.

Review Agent

Challenges the specification before implementation.

So instead of one coding agent doing everything:



This is much closer to **multi-agent engineering**.

---

12. The Specification Engineer's most important tool may be the "ambiguity detector"

I would actually make this a first-class capability.

Imagine the system reports:

```
SPECIFICATION ANALYSIS

Ambiguities: 7
Contradictions: 2
```

```
Untestable requirements: 4
Missing failure behavior: 11
Missing authorization rules: 3
Missing performance bounds: 6
Undefined terminology: 9
```

For example:

> "The system should respond quickly."

The tool asks:

> Define "quickly."
>
> Is this p50, p95, or p99?
>
> What workload?
>
> What payload size?
>
> What availability target?

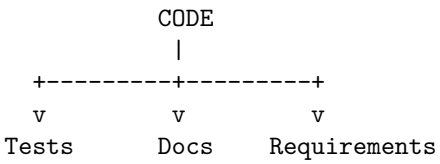
That turns **specification quality itself into something measurable**.

---

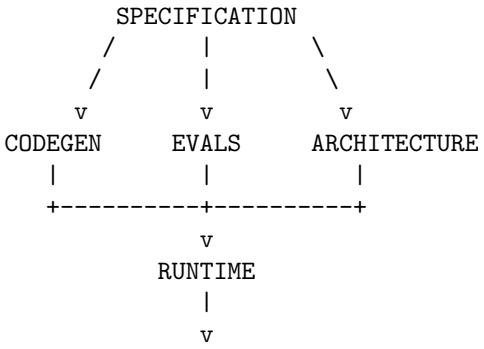
13. Eventually, specifications may become the primary engineering artifact

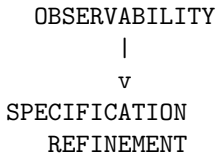
This is the deeper evolution I would predict.

Today:



Future:





Code becomes increasingly **derived**.

Specifications become increasingly **authoritative**.

That is the profound change.

---

14. I would define the future Specification Engineering stack like this

```
+-----------------------------------------------+
|              HUMAN INTENT                     |
+-----------------------------------------------+
| Specification elicitation                     |
| ambiguity detection                           |
| domain modeling                               |
+-----------------------------------------------+
|              SPECIFICATION                    |
|                                               |
| behavior | contracts | constraints | policies |
| state    | invariants | edge cases | failures |
+-----------------------------------------------+
|              DERIVATION                       |
|                                               |
| architecture | APIs | schemas | tests | evals |
+-----------------------------------------------+
|              AI ENGINEERING                   |
|                                               |
| coding agents | test agents | review agents   |
+-----------------------------------------------+
|              VERIFICATION                     |
|                                               |
| tests | formal checks | evals | security      |
+-----------------------------------------------+
|              RUNTIME                          |
|                                               |
| telemetry | traces | failures | user feedback |
+-----------------------------------------------+
|              LEARNING                         |
|                                               |
```

```
| specification refinement                      |
+-----------------------------------------------+
```

The three evolutionary stages

I'd summarize the evolution as:

Today: Documentation Specifications are mostly documents that humans interpret.

Near term: Structured specification Specifications become machine-readable and generate contracts, tests, evals, and implementation scaffolding.

Long term: Executable specification Specifications become **active system artifacts** that constrain agents, generate implementations, drive verification, monitor production behavior, and evolve from observed failures.

That leads to a very different definition of the engineer:

> **The traditional software engineer primarily transforms specifications into code. The AI-native Specification Engineer transforms intent into specifications that machines can implement and verify.**

And I think this is potentially one of the most important new roles in the AI-native software engineering stack—because as the marginal cost of generating code approaches zero, **the bottleneck moves from implementation to precise specification, verification, and control.**